Numerical Methods for ODEs

Analysis of a Strongly Damped Oscillator

Submitted by

SANJEEV KUMAWAT

Roll Number: ph25mscst11026

M.Sc. Physics

Instructor

ANUPAM GUPTA

Date of Submission

April 2, 2026

Contents

1 Introduction **2**

2 System **2**
2.1 Equations of Motion 2
2.2 Initial Conditions and Parameters 2

3 Numerical Methods **3**
3.1 Explicit Euler Method 3
3.2 Runge-Kutta 4 (RK4) 3
3.3 Implicit Euler Method 3
3.4 Crank-Nicolson Method 3

4 Stability Analysis **4**
4.1 Explicit Methods 4
4.2 Implicit Method Euler Method 5

5 Results **6**
5.1 Task 1 - Explicit Schemes 6
5.2 Task 2 - Stability of explicit methods 7
5.3 Task 3 - Implicit and Crank-Nicholson 9
5.4 The Stiffness Transition 11

6 Conclusion **12**

A Python Code **13**
A.1 Numerical Methods 13
A.2 System 15
A.3 Task - 1 15
A.4 Task - 2 18
A.5 Task - 3 19
A.6 The Stiff Transition 21

1 Introduction

Many physical systems involve processes that evolve on different timescales. When fast and slow dynamics coexist, the system becomes stiff, making numerical integration challenging because stability requires very small timesteps.
In this assignment, we study a strongly damped harmonic oscillator as a model system to understand stiffness. The equation is converted into a first-order system and solved using explicit methods, implicit methods, and the Crank–Nicolson scheme.
The aim is to compare these methods in terms of stability and accuracy, and to observe how stiffness affects their performance. By varying the damping coefficient, we also examine how the system transitions from non-stiff to stiff behaviour.
This study helps in understanding how numerical methods handle systems with multiple timescales and the trade-offs involved in their implementation.

2 System

2.1 Equations of Motion

The system under investigation is a one-dimensional damped harmonic oscillator of unit mass, governed by the second-order ordinary differential equation:

$$\ddot{x}(t) + \gamma\dot{x}(t) + \omega^2 x(t) = 0 \tag{1}$$

where $x(t)$ denotes the displacement, $\dot{x}(t)$ the velocity, $\omega = 1$ the natural frequency, and γ the viscous damping coefficient.
The assignment focuses on the strongly damped regime, $\gamma \gg \omega$, with the canonical parameter choice $\gamma = 20$.

To solve 2nd order differential using the numerical method it reformed as a first order system: Introduce the velocity $v = \dot{x}$. Thus, the equation becomes:

$$\dot{x} = v \tag{2}$$

$$\dot{v} = -\gamma v - w^2 x \tag{3}$$

2.2 Initial Conditions and Parameters

The following values are used unless stated otherwise:

1. $x(0) = 1$
2. $v(0) = 0$
3. $\omega = 1$
4. $\gamma = 20$

All quantities are expressed in dimensionless units.

3 Numerical Methods

Explicit Implicit methods are used for time integration to solve the system.

3.1 Explicit Euler Method

The Explicit Euler method is a first-order numerical scheme used to solve ordinary differential equations of the form $\dot{y} = f(t, y)$. It updates the solution using the slope evaluated at the current time step:

$$y_{n+1} = y_n + h, f(t_n, y_n) \tag{4}$$

It is simple to implement but conditionally stable, making it unsuitable for stiff systems unless very small timesteps are used.

3.2 Runge-Kutta 4 (RK4)

The Runge-Kutta 4 method is a fourth-order explicit method that improves accuracy by taking a weighted average of slopes at intermediate points:

$$k_1 = f(t_n, y_n) \tag{5}$$

$$k_2 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right) \tag{6}$$

$$k_3 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_2\right) \tag{7}$$

$$k_4 = f(t_n + h, y_n + hk_3) \tag{8}$$

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4) \tag{9}$$

It provides high accuracy but remains conditionally stable and can still fail for stiff systems.

3.3 Implicit Euler Method

The Implicit Euler method is a first-order implicit scheme where the function is evaluated at the next time step:

$$y_{n+1} = y_n + h, f(t_{n+1}, y_{n+1}) \tag{10}$$

This method is unconditionally stable and well-suited for stiff systems, but it may introduce numerical dissipation, causing the solution to decay faster than expected.

3.4 Crank-Nicolson Method

The Crank-Nicolson method is a second-order implicit method that averages the slopes at the current and next time steps:

$$y_{n+1} = y_n + \frac{h}{2}\left[f(t_n, y_n) + f(t_{n+1}, y_{n+1})\right] \tag{11}$$

It is unconditionally stable and more accurate than the implicit Euler method, with reduced numerical dissipation, although slight oscillations may appear for large timesteps.

4 Stability Analysis

4.1 Explicit Methods

To analyze the stability of explicit methods, we consider the system in matrix form:

$$\frac{d}{dt}\begin{bmatrix} x \\ v \end{bmatrix} = M \begin{bmatrix} x \\ v \end{bmatrix}$$

where

$$M = \begin{bmatrix} 0 & 1 \\ -\omega^2 & -\gamma \end{bmatrix}$$

The stability of the system is determined by the eigenvalues of the matrix M, obtained from:

$$\det(M - \lambda I) = 0$$

Solving, we get:

$$\lambda = \frac{-\gamma \pm \sqrt{\gamma^2 - 4\omega^2}}{2}$$

For $\gamma = 20$ and $\omega = 1$:

$$\lambda = \frac{-20 \pm \sqrt{396}}{2}$$

$$\lambda_1 \approx -0.05, \quad \lambda_2 \approx -19.95$$

These correspond to a slow decay mode (λ_1) and a fast decay mode (λ_2).
The presence of widely separated eigenvalues indicates that the system is stiff.
For the explicit Euler method:

$$y_{n+1} = (1 + h\lambda)y_n$$

Stability requires:

$$|1 + h\lambda| < 1$$

Using the fast mode $\lambda \approx -19.95$:

$$|1 - 19.95h| < 1$$

$$h < \frac{2}{19.95} \approx 0.1$$

Thus, the timestep must satisfy:

$$h < 0.1$$

This shows that stability is controlled by the fast decay mode, even though the physical behaviour is governed by the slow mode. This mismatch between numerical stability and physical timescales is the defining characteristic of stiffness.

4.2 Implicit Method Euler Method

The backward Euler method is given by

$$y_{n+1} = y_n + hf(x_{n+1}, y_{n+1}).$$

Consider this method for the ODE

$$\frac{dy}{dx} = -\lambda y, \qquad y(0) = 1, \qquad x \geq 0, \quad \lambda > 0.$$

Then the method becomes

$$y_{n+1} = y_n - h\lambda y_{n+1}.$$

Rearranging,

$$(1 + h\lambda)\, y_{n+1} = y_n$$

$$\Rightarrow \quad y_{n+1} = \frac{1}{1 + h\lambda}\, y_n.$$

For stability, we require

$$\left| \frac{1}{1 + h\lambda} \right| < 1,$$

which holds for all $h > 0$ when $\lambda > 0$.
Thus, the backward Euler method is **absolutely stable**.

5 Results

5.1 Task 1 - Explicit Schemes

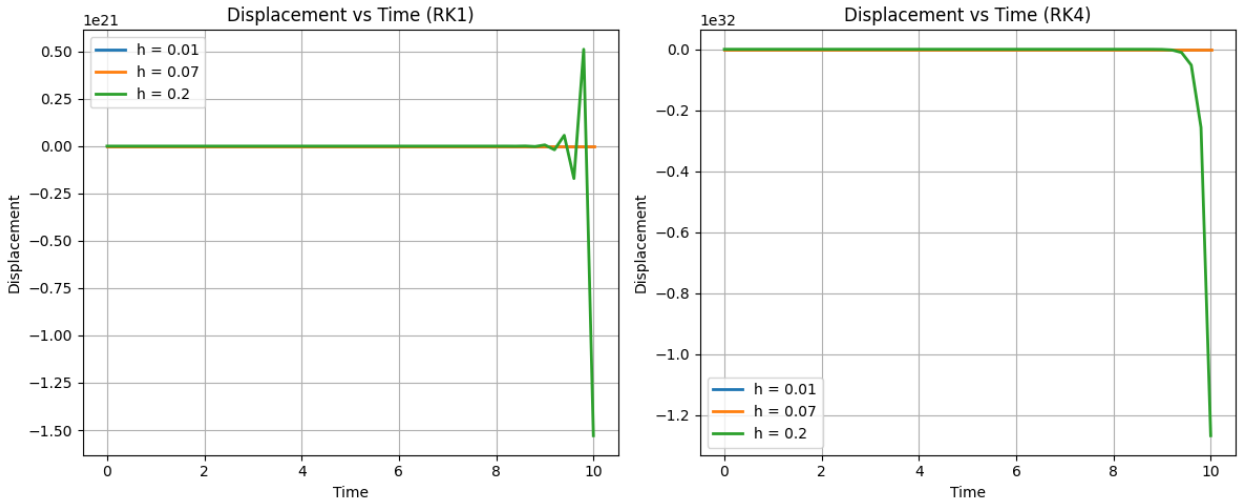

Figure 1

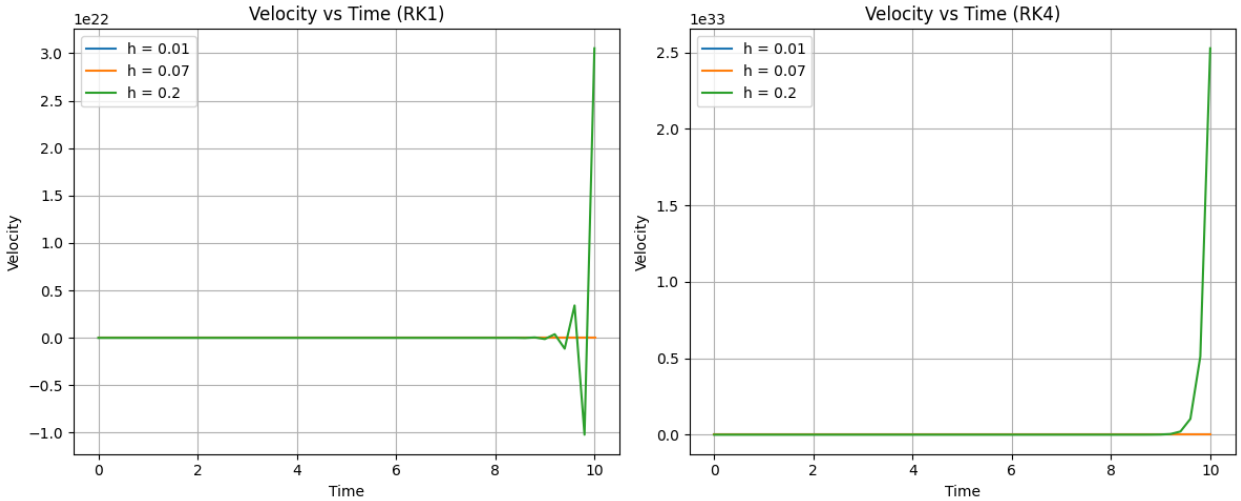

Figure 2

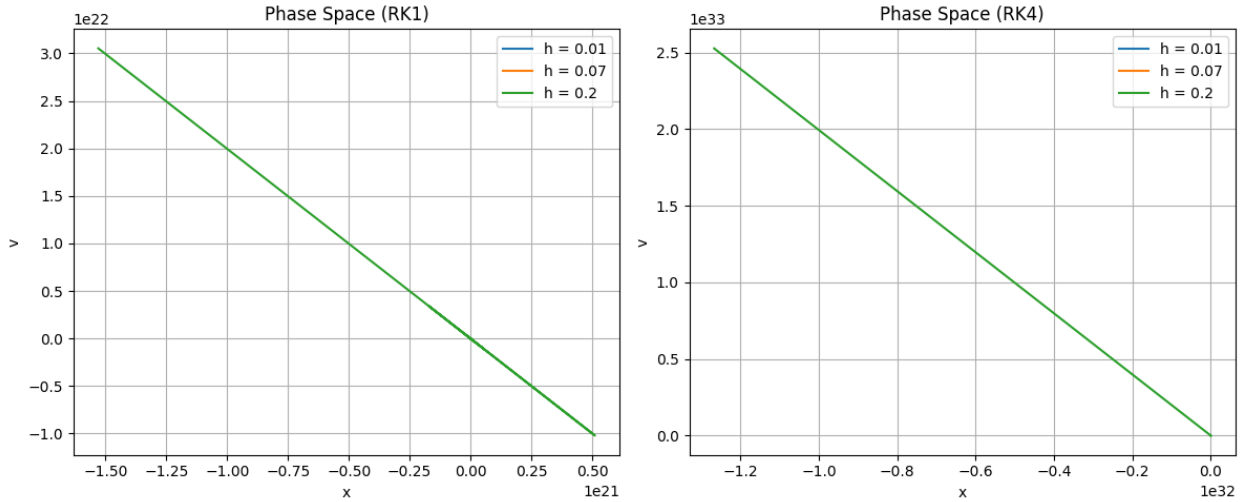

Figure 3

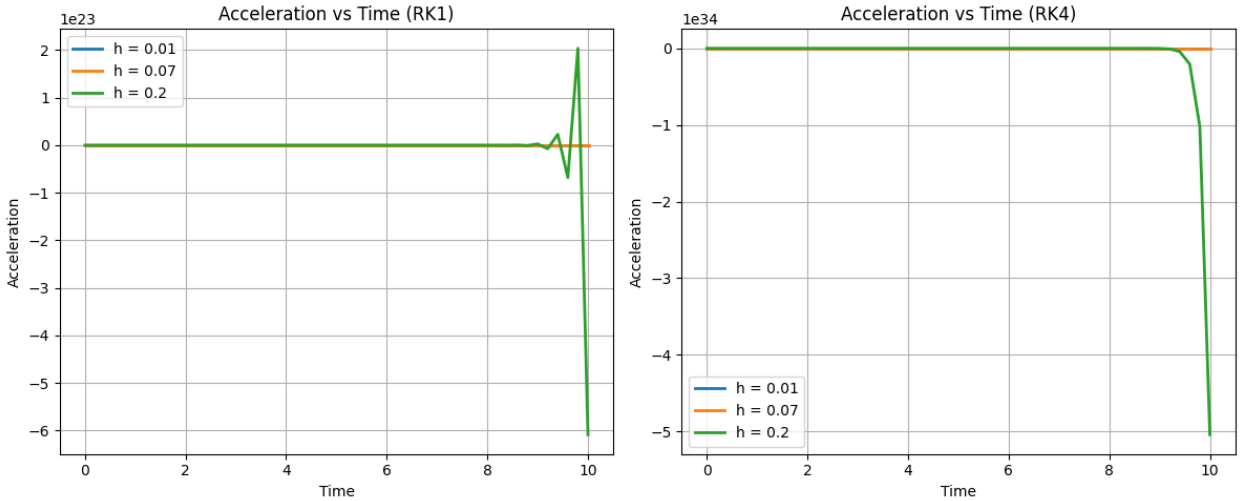

Figure 4

The results show that explicit methods are highly sensitive to timestep size. For small timesteps, both RK1 and RK4 accurately capture the overdamped behaviour. However, as the timestep increases, numerical errors grow and eventually lead to instability.
Higher-order methods like RK4 improve accuracy but do not eliminate the stability restriction. The appearance of oscillations and divergence for large timesteps highlights that stability is governed by numerical constraints rather than physical behaviour.
Thus, explicit methods require sufficiently small timesteps to remain stable, especially in systems with strong damping.

5.2 Task 2 - Stability of explicit methods

```
Eigenvalues:
lamda1: -0.050125628933798794
lamda2: -19.9498743710662

Euler's method is stable for the given ODE if
time step(h) <= 0.10025125786760089

Time scale for the fast decay mode: 0.050125628933800445
Time scale for the slow decay mode: 19.94987437106686
```

The stability of explicit methods was analyzed using the eigenvalues of the system matrix.The fast mode imposes a strict restriction on the timestep for stability. For explicit Euler, the stability condition requires ($h < 0.1$).
The timescale of the slow physical decay is determined by the smallest magnitude eigenvalue of the system. For the given system, the slow eigenvalue is $\lambda \approx -0.05$, leading to a timescale

$$\tau_{slow} = \frac{1}{|\lambda|} \approx 20$$

Comparing this with the stability-limited timestep $h \approx 0.1$, it is clear that the timestep required for numerical stability is much smaller than the physical timescale of interest.
This mismatch between stability requirements and physical timescales is the defining feature of stiffness.
Inshort, Stability is controlled by the fast mode, while physics is governed by the slow mode, leading to stiffness.

5.3 Task 3 - Implicit and Crank-Nicholson

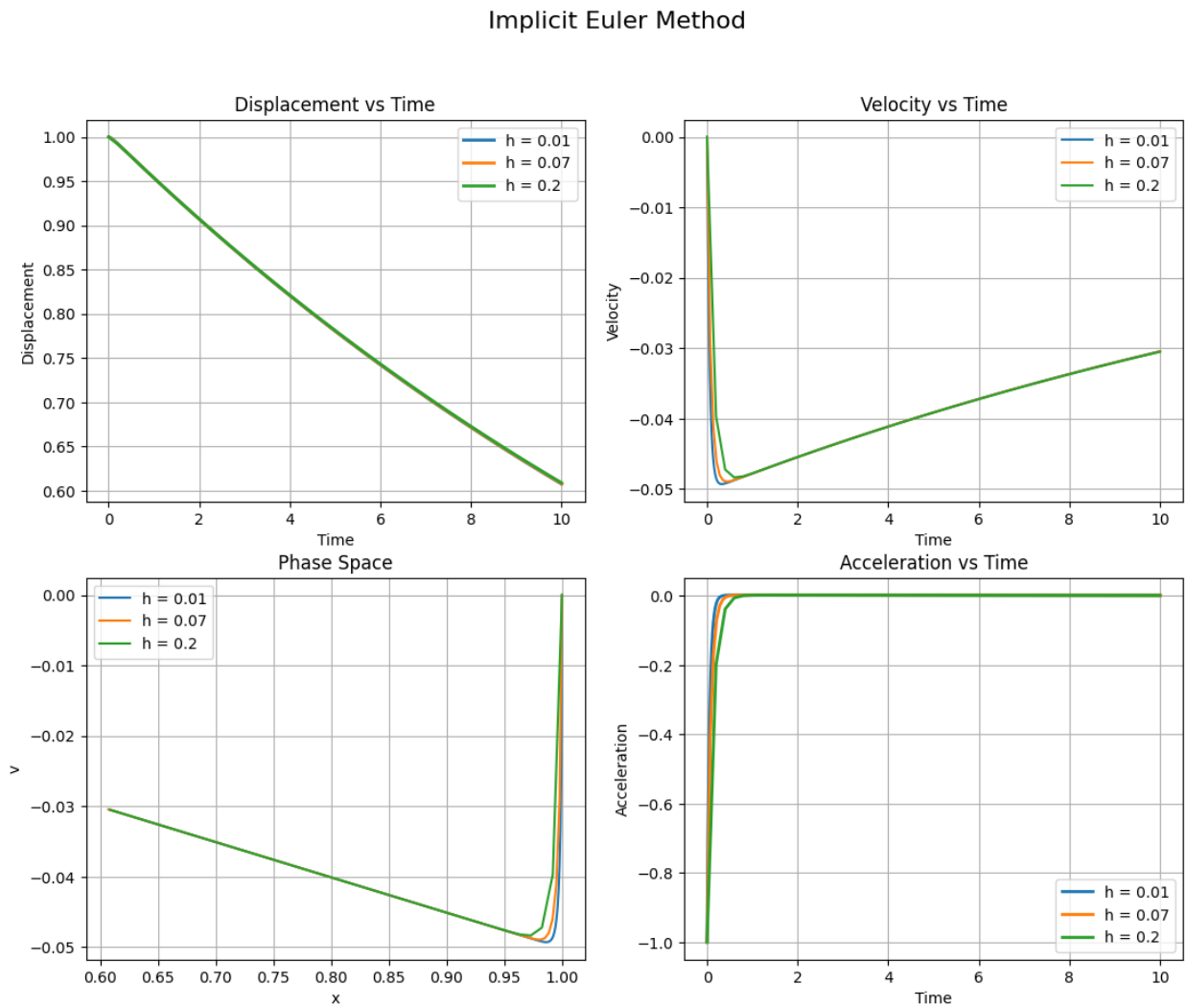

Figure 5

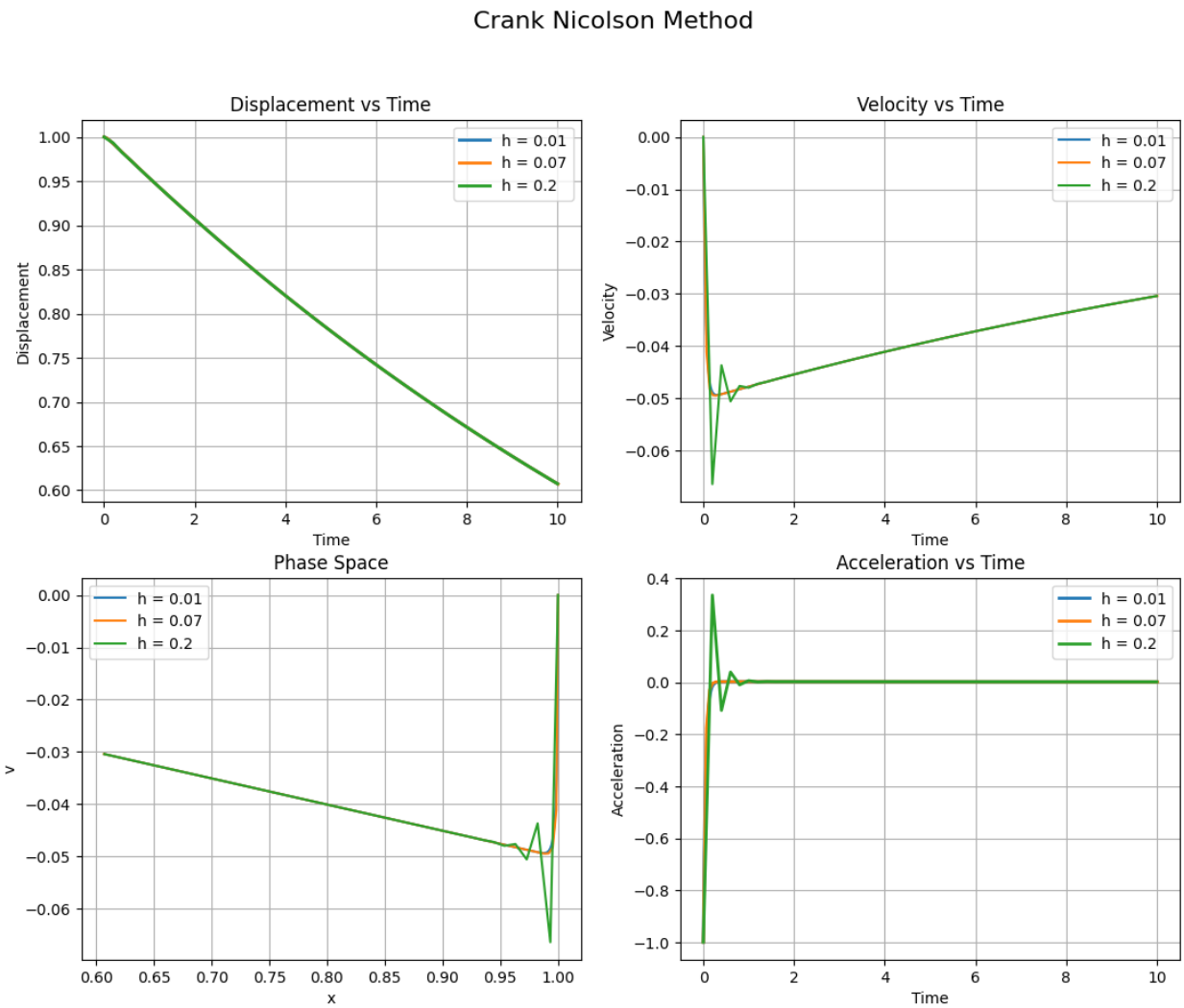

Figure 6

The implicit Euler and Crank–Nicolson methods were applied using timestep values that made explicit methods unstable. Both methods remained stable for all timesteps, showing unconditional stability.

The implicit Euler method produced smooth decay without oscillations, but for larger timesteps the solution decayed faster than expected. This happens due to numerical dissipation, where the method adds extra damping and makes the system overly damped.

In contrast, the Crank–Nicolson method gave results closer to the true physical behaviour. The decay was more accurate, and the phase-space plots showed slight oscillations for larger timesteps, indicating reduced numerical damping.

Overall, implicit Euler ensures stability by overdamping the system, while Crank–Nicolson maintains a better balance between stability and accuracy by preserving the system's dynamics.

5.4 The Stiffness Transition

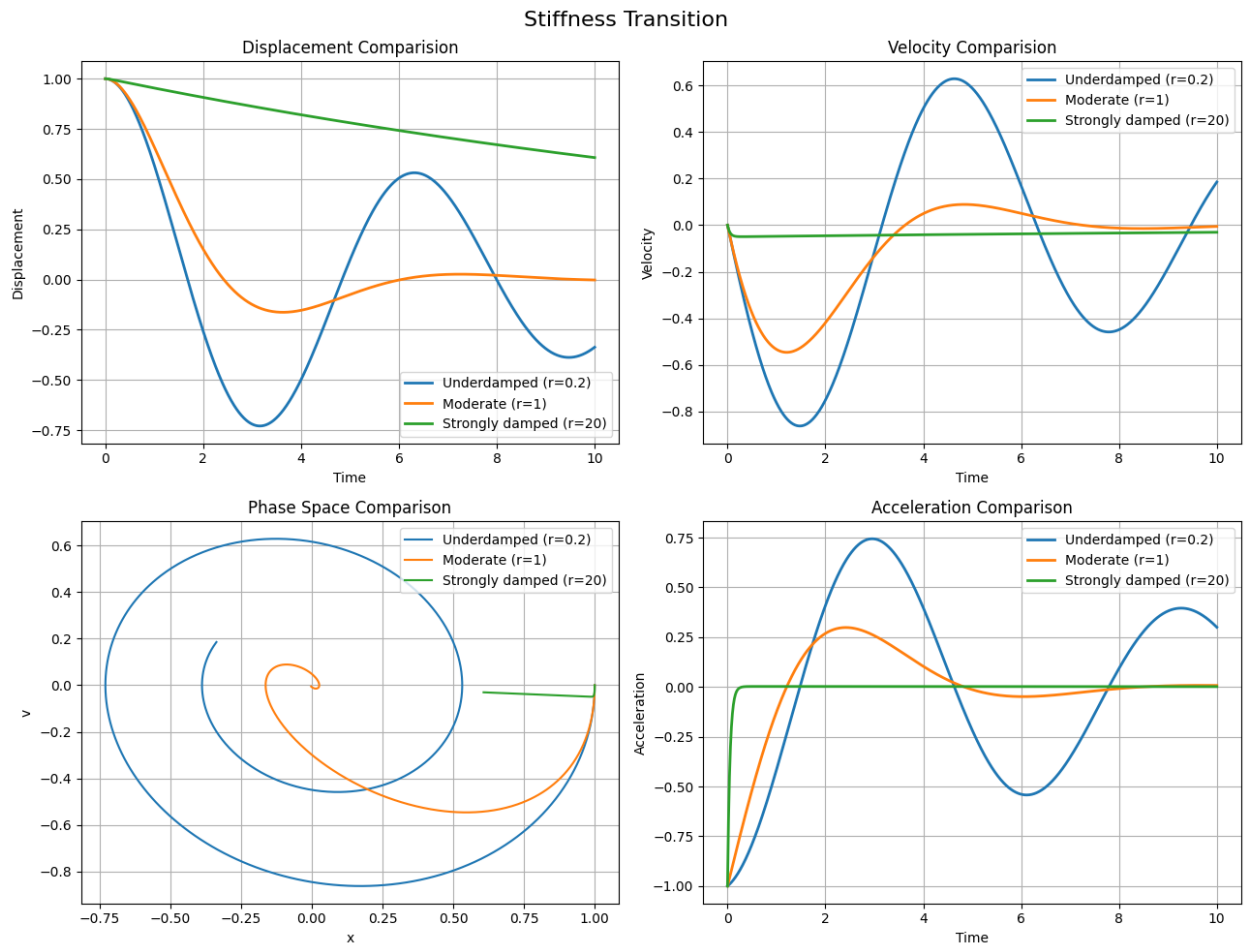

Figure 7

The stiffness transition was studied by changing the damping coefficient for three cases: underdamped ($r = 0.2$), moderate ($r = 1$), and strongly damped

$$(r = 20)$$

.

For the underdamped case, the system shows oscillations that slowly decrease with time. This means the system is not stiff. In the moderate case, the oscillations reduce and the system reaches equilibrium faster, showing a transition in behaviour.

In the strongly damped case, oscillations disappear completely. The system first decays very quickly and then relaxes slowly. This shows that two different timescales are present in the system.

From this comparison, it is clear that stiffness appears when the damping becomes large and creates a big difference between fast and slow dynamics. Even though the slow behaviour is physically important, numerical methods are forced to follow the fast part as well.

Thus, stiffness is not just due to large damping, but because of the presence of multiple timescales in the system.

6 Conclusion

In this study, explicit methods were found to become unstable for large timesteps due to the fast decay mode, even though the physical behaviour is governed by slower dynamics. Implicit methods overcome this issue, with implicit Euler providing stability through extra damping, while Crank–Nicolson maintains better accuracy with less numerical dissipation. The stiffness transition showed that stiffness arises from the presence of multiple timescales in the system. Overall, the choice of method depends on the balance between stability and accuracy in stiff systems.

A Python Code

Imports

```
#importing required libraries
import numpy as np
import matplotlib.pyplot as plt
```

A.1 Numerical Methods

```
#Defining the methods as function to solve the given system problem

#explicit methods
def rk1(f, x0, y0, h, n, w, r):
    x = x0
    y = np.array(y0)
    x_values = [x0]
    y_values = [y.copy()]

    for i in range(n):
        y = y + h * f(x, y, w, r)
        x = x + h
        x_values.append(x)
        y_values.append(y.copy())

    return np.array(x_values), np.array(y_values)


def rk2(f, x0, y0, h, n, w, r):
    x = x0
    y = np.array(y0)
    x_values = [x0]
    y_values = [y.copy()]

    for i in range(n):
        k1 = f(x,y, w,r)
        k2  = f(x + h, y + h * k1)
        y = y + ((h/2) * (k1 + k2))
        x = x + h
        x_values.append(x)
        y_values.append(y.copy())

    return np.array(x_values), np.array(y_values)


def rk4(f, x0, y0, h, n, w, r):
    x = x0
```

```
    y = np.array(y0)
    x_values = [x0]
    y_values = [y.copy()]

    for i in range(n):
        k1 = f(x, y, w, r)
        k2 = f(x + h/2, y + (h/2)*k1, w, r)
        k3 = f(x + h/2, y + (h/2)*k2, w, r)
        k4 = f(x + h, y + h*k3, w, r)
        y = y + (h/6)*(k1 + 2*k2 + 2*k3 + k4)
        x = x + h
        x_values.append(x)
        y_values.append(y.copy())

    return np.array(x_values), np.array(y_values)




#implicit methods

#backward euler method
def backward_euler_method(f, x0, y0, h, n, w, r):
    M = np.array([[0,1],
        [-w**2, -r]])
    I = np.eye(2)
    inv_mat = np.linalg.inv(I - h*M)

    x = x0
    y = np.array(y0)
    x_values = [x0]
    y_values = [y.copy()]

    for i in range(n):
        y = inv_mat @ y
        x = x + h

        x_values.append(x)
        y_values.append(y.copy())

    return np.array(x_values), np.array(y_values)

#Cranck-Nicolson Method
def crank_nicolson_method(f, x0, y0, h, n, w, r):
  x = x0
  y = np.array(y0)
  x_values = [x0]
```

```
  y_values = [y.copy()]

  M = np.array([[0,1],
        [-w**2, -r]])
  I = np.eye(2)
  mat1 = np.linalg.inv(I - (h/2)*M)
  mat2 = I + (h/2)*M

  for i in range(n):
      y = mat1 @ (mat2 @ y)
      x = x+h
      x_values.append(x)
      y_values.append(y.copy())

  return np.array(x_values), np.array(y_values)
```

A.2 System

```
#the ODE function
def f(t, y, w,r):
    x = y[0]    #position
    v = y[1]    #velocity
    dv = -r*v - (w**2)*x

    return np.array([v, dv]) #velocity, accelaration
#Initial Parameters
t0 = 0
x0 = 1
v0 = 0
w = 1
r = 20
tf = 10 #final time
y0 = np.array([x0, v0])

#Taking three different timesteps values
time_steps = [0.01, 0.07, 0.2]
n_values = []

for h in time_steps:
    n = int(np.ceil((tf - t0) / h))
    n_values.append(n)
```

A.3 Task - 1

```
#displacement plots
fig, ax = plt.subplots(1, 2, figsize=(12,5))
```

```
#RK1
for h, n in zip(time_steps, n_values):
    t_vals, y_vals = rk1(f, t0, y0, h, n, w, r)
    x_vals = y_vals[:, 0]
    ax[0].plot(t_vals, x_vals, linewidth=2, label=f"h = {h}")

ax[0].set_title("Displacement vs Time (RK1)", fontsize=12)
ax[0].set_xlabel("Time")
ax[0].set_ylabel("Displacement")
ax[0].legend()
ax[0].grid()


#RK4
for h, n in zip(time_steps, n_values):
    t_vals, y_vals = rk4(f, t0, y0, h, n, w, r)
    x_vals = y_vals[:, 0]
    ax[1].plot(t_vals, x_vals, linewidth=2, label=f"h = {h}")

ax[1].set_title("Displacement vs Time (RK4)", fontsize=12)
ax[1].set_xlabel("Time")
ax[1].set_ylabel("Displacement")
ax[1].legend()
ax[1].grid()

plt.tight_layout()
plt.show()


#velocity plots
fig, ax = plt.subplots(1, 2, figsize=(12,5))

#RK1
for h, n in zip(time_steps, n_values):
    t_vals, y_vals = rk1(f, t0, y0, h, n, w, r)
    v_vals = y_vals[:, 1]
    ax[0].plot(t_vals, v_vals, label=f"h = {h}")

ax[0].set_title("Velocity vs Time (RK1)")
ax[0].set_xlabel("Time")
ax[0].set_ylabel("Velocity")
ax[0].legend()
ax[0].grid()

#RK4
for h, n in zip(time_steps, n_values):
```

```
    t_vals, y_vals = rk4(f, t0, y0, h, n, w, r)
    v_vals = y_vals[:, 1]
    ax[1].plot(t_vals, v_vals, label=f"h = {h}")

ax[1].set_title("Velocity vs Time (RK4)")
ax[1].set_xlabel("Time")
ax[1].set_ylabel("Velocity")
ax[1].legend()
ax[1].grid()

plt.tight_layout()
plt.show()


#phase space plots
fig, ax = plt.subplots(1, 2, figsize=(12,5))

#RK1
for h, n in zip(time_steps, n_values):
    t_vals, y_vals = rk1(f, t0, y0, h, n, w, r)
    ax[0].plot(y_vals[:,0], y_vals[:,1], label=f"h = {h}")

ax[0].set_title("Phase Space (RK1)")
ax[0].set_xlabel("x")
ax[0].set_ylabel("v")
ax[0].legend()
ax[0].grid()

#RK4
for h, n in zip(time_steps, n_values):
    t_vals, y_vals = rk4(f, t0, y0, h, n, w, r)
    ax[1].plot(y_vals[:,0], y_vals[:,1], label=f"h = {h}")

ax[1].set_title("Phase Space (RK4)")
ax[1].set_xlabel("x")
ax[1].set_ylabel("v")
ax[1].legend()
ax[1].grid()

plt.tight_layout()
plt.show()


#accelaration graphs
fig, ax = plt.subplots(1, 2, figsize=(12,5))

#RK1
```

```
for h, n in zip(time_steps, n_values):
    t_vals, y_vals = rk1(f, t0, y0, h, n, w, r)

    acc_vals = np.zeros((len(t_vals)))
    for i in range(len(t_vals)):
        v, acc_vals[i] = f(t_vals[i], y_vals[i], w, r)
    ax[0].plot(t_vals, acc_vals, linewidth=2, label=f"h = {h}")
ax[0].set_title("Acceleration vs Time (RK1)")
ax[0].set_xlabel("Time")
ax[0].set_ylabel("Acceleration")
ax[0].legend()
ax[0].grid()

#RK4
for h, n in zip(time_steps, n_values):
    t_vals, y_vals = rk4(f, t0, y0, h, n, w, r)

    acc_vals = np.zeros((len(t_vals)))
    for i in range(len(t_vals)):
        v, acc_vals[i] = f(t_vals[i], y_vals[i], w, r)
    ax[1].plot(t_vals, acc_vals, linewidth=2, label=f"h = {h}")

ax[1].set_title("Acceleration vs Time (RK4)")
ax[1].set_xlabel("Time")
ax[1].set_ylabel("Acceleration")
ax[1].legend()
ax[1].grid()


plt.tight_layout()
plt.show()
```

A.4 Task - 2

```
M = np.array([[0,1],
        [-w**2, -r]])
eig_vals = np.linalg.eigvals(M)

#time scale for the fast and slow decay mode
time_scale_1 = 1 / np.max(np.abs(eig_vals)) #fast mode
time_scale_2 = 1 / np.min(np.abs(eig_vals)) #slow mode

#largest time step
h = 2/(np.max(np.abs(eig_vals)))

#print the results
```

```
print(f"Eigenvalues:\nlamda1: {eig_vals[0]}\nlamda2: {eig_vals[1]}\n
   ")
print(f"Euler's method is stable for the given ODE if time step(h)
   <= {h}\n")
print(f"Time scale for the fast decay mode: {time_scale_1}")
print(f"Time scale for the slow decay mode: {time_scale_2}")
```

A.5 Task - 3

```
#Implicit Euler Method

fig, ax = plt.subplots(2, 2, figsize=(13, 10))
fig.suptitle('Implicit Euler Method', fontsize=16)

#displace plots
for h, n in zip(time_steps, n_values):
    t_vals, y_vals = backward_euler_method(f, t0, y0, h, n, w, r)
    x_vals = y_vals[:, 0]
    ax[0,0].plot(t_vals, x_vals, linewidth=2, label=f"h = {h}")

ax[0,0].set_title("Displacement vs Time", fontsize=12)
ax[0,0].set_xlabel("Time")
ax[0,0].set_ylabel("Displacement")
ax[0,0].legend()
ax[0,0].grid()

#velocity plots
for h, n in zip(time_steps, n_values):
    t_vals, y_vals = backward_euler_method(f, t0, y0, h, n, w,r)
    v_vals = y_vals[:, 1]
    ax[0,1].plot(t_vals, v_vals, label=f"h = {h}")

ax[0,1].set_title("Velocity vs Time")
ax[0,1].set_xlabel("Time")
ax[0,1].set_ylabel("Velocity")
ax[0,1].legend()
ax[0,1].grid()

#phase space plots
for h, n in zip(time_steps, n_values):
    t_vals, y_vals = backward_euler_method(f, t0, y0, h, n, w, r)
    ax[1,0].plot(y_vals[:,0], y_vals[:,1], label=f"h = {h}")

ax[1,0].set_title("Phase Space")
ax[1,0].set_xlabel("x")
ax[1,0].set_ylabel("v")
ax[1,0].legend()
```

```
ax[1,0].grid()

#accelaration graphs
for h, n in zip(time_steps, n_values):
    t_vals, y_vals = backward_euler_method(f, t0, y0, h, n, w, r)

    acc_vals = np.zeros((len(t_vals)))
    for i in range(len(t_vals)):
        v, acc_vals[i] = f(t_vals[i], y_vals[i], w,r)
    ax[1,1].plot(t_vals, acc_vals, linewidth=2, label=f"h = {h}")
ax[1,1].set_title("Acceleration vs Time")
ax[1,1].set_xlabel("Time")
ax[1,1].set_ylabel("Acceleration")
ax[1,1].legend()
ax[1,1].grid()


#Crank Nicolson Method

fig, ax = plt.subplots(2, 2, figsize=(13, 10))
fig.suptitle('Crank Nicolson Method', fontsize=16)

#displace plots
for h, n in zip(time_steps, n_values):
    t_vals, y_vals = crank_nicolson_method(f, t0, y0, h, n, w, r)
    x_vals = y_vals[:, 0]
    ax[0,0].plot(t_vals, x_vals, linewidth=2, label=f"h = {h}")

ax[0,0].set_title("Displacement vs Time", fontsize=12)
ax[0,0].set_xlabel("Time")
ax[0,0].set_ylabel("Displacement")
ax[0,0].legend()
ax[0,0].grid()


#velocity plots
for h, n in zip(time_steps, n_values):
    t_vals, y_vals = crank_nicolson_method(f, t0, y0, h, n, w, r)
    v_vals = y_vals[:, 1]
    ax[0,1].plot(t_vals, v_vals, label=f"h = {h}")

ax[0,1].set_title("Velocity vs Time")
ax[0,1].set_xlabel("Time")
ax[0,1].set_ylabel("Velocity")
ax[0,1].legend()
ax[0,1].grid()
```

```
#phase space plots
for h, n in zip(time_steps, n_values):
    t_vals, y_vals = crank_nicolson_method(f, t0, y0, h, n, w, r)
    ax[1,0].plot(y_vals[:,0], y_vals[:,1], label=f"h = {h}")

ax[1,0].set_title("Phase Space")
ax[1,0].set_xlabel("x")
ax[1,0].set_ylabel("v")
ax[1,0].legend()
ax[1,0].grid()



#accelaration graphs
for h, n in zip(time_steps, n_values):
    t_vals, y_vals = crank_nicolson_method(f, t0, y0, h, n, w, r)

    acc_vals = np.zeros((len(t_vals)))
    for i in range(len(t_vals)):
        v, acc_vals[i] = f(t_vals[i], y_vals[i], w, r)
    ax[1,1].plot(t_vals, acc_vals, linewidth=2, label=f"h = {h}")
ax[1,1].set_title("Acceleration vs Time")
ax[1,1].set_xlabel("Time")
ax[1,1].set_ylabel("Acceleration")
ax[1,1].legend()
ax[1,1].grid()
```

A.6 The Stiff Transition

```
#Stiffness transition study

r_values = [0.2, 1, 20]   # underdamped, moderate, stiff

fig, ax = plt.subplots(2, 2, figsize=(13, 10))
fig.suptitle('Stiffness Transition', fontsize=16)

h = 0.01
n = int(np.ceil((tf - t0) / h))

for r in r_values:

    #redefine ODE with current r
    def f_local(t, y, w, r):
        x = y[0]
        v = y[1]
```

```
        return np.array([v, -r*v - (w**2)*x])

    t_vals, y_vals = rk4(f_local, t0, y0, h, n, w, r)

    acc_vals = np.zeros((len(t_vals)))
    for i in range(len(t_vals)):
        v, acc_vals[i] = f_local(t_vals[i], y_vals[i], w, r)

    if r == 0.2:
        label = "Underdamped (r=0.2)"
    elif r == 1:
        label = "Moderate (r=1)"
    else:
        label = "Strongly damped (r=20)"

    ax[0,0].plot(t_vals, y_vals[:,0], linewidth=2, label=label)
    ax[0,1].plot(t_vals, y_vals[:, 1], linewidth=2, label=label)
    ax[1,0].plot(y_vals[:,0], y_vals[:,1],label=label)
    ax[1,1].plot(t_vals, acc_vals, linewidth=2, label=label)

ax[0,0].set_title("Displacement Comparision", fontsize=12)
ax[0,0].set_xlabel("Time")
ax[0,0].set_ylabel("Displacement")
ax[0,0].legend()
ax[0,0].grid()

ax[0,1].set_title("Velocity Comparision")
ax[0,1].set_xlabel("Time")
ax[0,1].set_ylabel("Velocity")
ax[0,1].legend()
ax[0,1].grid()

ax[1,0].set_title("Phase Space Comparison")
ax[1,0].set_xlabel("x")
ax[1,0].set_ylabel("v")
ax[1,0].legend()
ax[1,0].grid()

ax[1,1].set_title("Acceleration Comparison")
ax[1,1].set_xlabel("Time")
ax[1,1].set_ylabel("Acceleration")
ax[1,1].legend()
ax[1,1].grid()

plt.tight_layout()
plt.show()
```

References

[1] Steve Brunton, "Numerical Stability for ODEs (Part 1)," YouTube lecture, 2020. Available at: https://www.youtube.com/watch?v=_zHlRpgZ3-0 [Accessed: 31 March 2026].

[2] Sanjeev K, "Python code for time integration techniques used in this report," GitHub repository, 2026. Available at: https://github.com/arsanjeevk/Computational_Methods/tree/main/Assignment-8(Time-integration-techniques) [Accessed: 2 April 2026].